Zero to Hero Study Handbook: LangGraph Multi-Agent Platform

This handbook is based on static analysis of the repository files in this project.

Module 1: Foundations & Architecture

1.1 What this project does

This project implements a production-oriented multi-agent workflow platform using LangGraph.

At a high level, it provides:

- A stateful, typed workflow engine for multi-step reasoning and report generation.
- A FastAPI backend with operational endpoints (`/workflow`, `/chat`, `/graph`, `/memory`, `/reports`, etc.).
- A Streamlit dashboard for graph inspection, workflow execution, memory browsing, and analytics.
- A Typer CLI (`langgraph-platform`) for command-line operations.
- Persistent memory in SQLite plus semantic retrieval via ChromaDB.
- RAG ingestion and retrieval for files and URLs.
- MCP-style tool exposure via `/mcp/capabilities` and `/mcp/call`.

Primary use cases supported by the code:

- Generate enterprise-style reports with citations and quality gates.
- Run multi-agent orchestration with planning, retrieval, writing, fact-checking, reflection, QA, and supervision.
- Ingest knowledge from documents/URLs into vector memory and retrieve it during workflow runs.
- Inspect system behavior through API metrics, dashboard charts, and persisted run history.

1.2 Core paradigms and patterns used in this codebase

A) Typed state machine orchestration (LangGraph)

- Implemented in `src/langgraph_platform/engine/workflow.py`.
- Uses `StateGraph(dict)` with explicit nodes and edges.
- Node handlers are pure-ish functions over a serialized state dict:
  - `planner_node`
  - `parallel_research_node`
  - `knowledge_merge_node`
  - `consensus_reasoning_node`
  - `writer_node`
  - `fact_checker_node`
  - `reflection_node`

  - `critic_node`
  - `qa_node`
  - `citation_node`
  - `supervisor_node`
  - `finalize_node`

Definition first: a state machine is a model where execution moves through named states (nodes) with explicit transition rules.

Here, each node reads and updates `WorkflowState` and returns a serialized dict for the next node.

B) Strong typing with Pydantic models

- Implemented in `src/langgraph_platform/state/models.py` and `src/langgraph_platform/config/settings.py`.
- `WorkflowState` captures all workflow data: request, routing flags, search results, retrieved docs, intermediate outputs, reports, citations, verification status, token usage, metadata, tool calls.
- `AppConfig` and nested config models enforce typed runtime configuration.

Definition first: typed models are explicit data contracts that validate structure and improve safety/documentation.

C) Hybrid style: object-oriented runtime + functional node handlers

- OOP classes hold infrastructure/stateful dependencies:
  - `LangGraphWorkflowEngine`
  - `SQLiteStore`
  - `ChromaMemoryStore`
  - `RAGPipeline`
  - `SystemMonitor`
  - `AnalyticsService`
- Node functions in `engine/nodes.py` operate functionally over state snapshots.

Definition first: this hybrid pattern combines OOP for lifecycle/resource management and functional composition for deterministic flow logic.

D) Registry/plugin pattern for tools and agents

- Tool abstraction uses `Tool` protocol and `ToolRegistry` (`src/langgraph_platform/tools/base.py`).
- Built-ins are registered in `build_default_registry` (`src/langgraph_platform/tools/builtin.py`).
- Agent specs are centralized in `AGENT_REGISTRY` (`src/langgraph_platform/agents/registry.py`).
- Plugin contracts exist in `src/langgraph_platform/plugins/base.py` and discovery in `plugins/registry.py`.

Definition first: registry pattern centralizes component discovery by name and decouples callers from concrete implementations.

E) Layered config loading with environment overrides

- YAML loading/merge: `src/langgraph_platform/config/loader.py`.
- Typed model validation and env override map: `src/langgraph_platform/config/settings.py`.

Definition first: layered config allows base defaults + specialized overlay files + environment-specific overrides.

1.3 Architecture: components and interactions

Main component boundaries:

- `engine/`: workflow graph construction and execution.
- `state/`: typed contracts for graph state and result objects.
- `agents/`: role/objective/tool/output contracts for 20 agents.
- `tools/`: pluggable tool implementations (search, docs, SQL, conversion, file readers, etc.).
- `memory/`: SQLite persistence and Chroma semantic store.
- `rag/`: ingestion and retrieval pipeline.
- `api/`: FastAPI endpoints and request/response schemas.
- `ui/`: Streamlit pages and graph visualization helpers.
- `cli/`: Typer command surface.
- `monitoring/` + `analytics/`: system metrics and run analytics.
- `mcp/`: MCP-style API adapter and external client abstraction.

ASCII main flow (from `engine/workflow.py` and `docs/workflow.mmd`):

```
START
  |
  v
planner
  |
  v
parallel_research
  |
  v
knowledge_merge
  |
  v
consensus_reasoning
  |
  v
writer
  |
  v
fact_checker
  |
  v
```

```
reflection
  |
  v
critic
  |
  v
qa
  |
  v
citation
  |
  v
supervisor
  |\
  | \-- retry --> writer
  |
  \---- finalize --> finalize --> END
```

How data and services connect:

```
CLI / API / Streamlit
      |
      v
LangGraphWorkflowEngine
      |
      +--> NodeRuntime
      |      +--> ToolRegistry (built-ins)
      |      +--> OllamaClient
      |      +--> SystemMonitor
      |
      +--> SQLiteStore (workflow_runs, graph_states, ...)
      +--> ChromaMemoryStore (collection: knowledge)
      +--> RAGPipeline (load -> chunk -> store -> retrieve)
```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`pyproject.toml`	Project metadata, dependencies, CLI entry point	`project.scripts: langgraph-platform = langgraph_platform.cli:app`	`requires-python`, dependency list, Ruff/Pytest settings
`configs/config.yaml`	Base runtime config	N/A	`app`, `model`, `retry`, `routing`, `memory`, `mcp`, `monitoring`, `tools`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`configs/*/default.yaml`	Layered config overlays	N/A	Graph/routing/retries/memory/embeddings/chun
`.env.example`	Environment variable examples	N/A	`OLLAMA_*`, `PLATFORM_SQLITE_PATH`, API keys
`src/langgraph_platform/config/settings.py`	Typed config models + env override logic	`AppConfig`, `ModelConfig`, `apply_env_overrides`	`env_map` with `OLLAMA_PLANNER_MODEL`, `PLATFORM_SQLITE_PATH`, etc.
`src/langgraph_platform/config/loader.py`	YAML loading + merge + validation	`load_config`, `_merge_dict`	Loads `configs/config.yaml` then merges all `**/*.yaml`
`src/langgraph_platform/models/state.py`	Shared typed workflow state contracts	`WorkflowState`, `ExecutionMetadata`, `WorkflowResult`, `RoutingDecision`	`VerificationStatus`, `NodeStatus`, `HITLAction` enums
`src/langgraph_platform/agents/registry.py`	Agent definitions	`AGENT_REGISTRY`, `get_agent`, `list_agents`	Agent tools, constraints, output schemas
`src/langgraph_platform/prompts/agents.py`	Prompt templates for planner/research/writer/ver	`PLANNER_PROMPT`, `RESEARCH_PROMPT`, ification/supervisor	JSON output schema instructions in prompt text
`src/langgraph_platform/engine/workflow.py`	Graph construction, execution, retry routing, run output	`LangGraphWorkflowEngine`, `_build_graph`, `run`, `inspect_graph`	`max_retries`, `retry.confidence_threshold`, optional `ENABLE_MLFLOW`
`src/langgraph_platform/engine/nodes.py`	Node implementation logic	`planner_node`, `parallel_research_node`, `writer_node`, ...	Uses `runtime.config.*`, sets `state.intermediate_outputs`, updates `state.confidence_score`
`src/langgraph_platform/engine/routers.py`	Routing decisions	`decide_routing`, `should_retry_reflection`	Keyword lists from `RoutingConfig`
`src/langgraph_platform/clients/llm.py`	Ollama client + fallback strategy	`OllamaClient`, `generate_with_fallback`, `json_with_fallback`	`OLLAMA_TIMEOUT_SECONDS`, `REQUIRE_LIVE_LLM`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
src/langgraph_platform/tools/base.py	Tool protocol and registry	Tool, ToolResult, ToolRegistry	ToolContext fields workflow_id, session_id
src/langgraph_platform/tools/builtin.py	Built-in tools and registry assembly	DuckDuckGoSearchTool, GitHubSearchTool, MemorySearchTool, build_default_registry	Tool names (duckduckgo_search, chroma_search, sql_tool, etc.)
src/langgraph_platform/memory/sqlite_store.py	Persistent operational store	SQLiteStore, ORM models (WorkflowRun, GraphStateRecord, ...)	SQLite URL from AppConfig.sqlite_url
src/langgraph_platform/memory/vector_store.py	Chroma vector storage abstraction	ChromaMemoryStore, add_documents, search	Collection name knowledge, optional USE_SENTENCE_TRANSFORMERS
src/langgraph_platform/rag/loaders.py	Load content from files/URLs	DocumentLoader.load_paths, load_url	Supports .md, .txt, .pdf, .csv, .json, .html
src/langgraph_platform/rag/pipeline.py	RAG ingest/retrieve orchestration	RAGPipeline.ingest_paths, ingest_urls, retrieve	chunk_size, chunk_overlap, top_k
src/langgraph_platform/api/schemas.py	Request/response contracts	ChatRequest, WorkflowRequest, HITLRequest, etc.	Pydantic field constraints (e.g., min_length=1)
src/langgraph_platform/api/main.py	FastAPI app and endpoint logic	create_app and endpoint handlers	Exposes /chat, /workflow, /graph, /memory, /reports, /mcp/*
apps/fastapi/main.py	Uvicorn import target	app import alias	Used by uvicorn apps.fastapi.main:app
src/langgraph_platform/cli/app.py	CLI command registration	run_workflow, graph_info, state, memory, report, doctor, dashboard	Typer command names and argument shapes

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`src/langgraph_platform/ui/dashboard.py`	Streamlit multipage UI	`launch_dashboard`	Sidebar pages: Dashboard, Workflow Graph, Live Execution, etc.
`src/langgraph_platform/graph_visualization.py`	Graph conversion/export utilities	`visualize_graph`, `to_graphviz_dot`, `export_graph_files`	Writes to `artifacts/graphs`
`src/langgraph_platform/monitoring/system.py`	Runtime metrics + node timing	`SystemMonitor.capture`, `start_node`, `stop_node`	Optional GPU via NVML
`src/langgraph_platform/analytics/summary.py`	Aggregated Plotly charts over run history	`confidence_trend`, `status_distribution`	Pulls data from `SQLiteStore.list_recent_runs`
`src/langgraph_platform/reporting/exporter.py`	Markdown/HTML/PDF/ISON report exports	`ReportExporter`	Output directory default: `artifacts/reports`
`src/langgraph_platform/mcp/server.py`	MCP-style internal tool exposure	`MCPServerAdapter`	Exposes selected tools over `/mcp/capabilities`, `/mcp/call`
`src/langgraph_platform/hitl/service.py`	Human-in-the-loop action state	`HITLService.apply`, `HITLState`	Tracks `approved`, `paused`, `rejected`, `overrides` by workflow ID

Suggested first-read files for new contributors:

1. `src/langgraph_platform/engine/workflow.py`
2. `src/langgraph_platform/engine/nodes.py`
3. `src/langgraph_platform/state/models.py`
4. `src/langgraph_platform/api/main.py`
5. `src/langgraph_platform/tools/builtin.py`

Module 3: Core Execution Flows

3.1 Flow A: CLI end-to-end workflow run

Entry path:

- `pyproject.toml` defines `langgraph-platform = langgraph_platform.cli.app:app`.
- Typer command: `run` in `src/langgraph_platform/cli/app.py`.

Key function path:

1. `run_workflow(request: str)`
2. `config = load_config()`
3. `engine = LangGraphWorkflowEngine(config)`
4. `result = engine.run(request)`
5. Print:
   - `workflow_id`
   - `confidence`
   - `final_report`
6. `engine.close()` in `finally`

Short source fragment:

```
result = engine.run(request)
print(f"workflow_id: {result.workflow_id}")
print(f"confidence: {result.confidence:.2f}")
print(result.final_report)
```

`WorkflowResult` output shape (`state/models.py`):

- `workflow_id: str`
- `final_report: str`
- `confidence: float`
- `verification_status: VerificationStatus`
- `citations: list[Citation]`
- `metadata: ExecutionMetadata`

3.2 Flow B: FastAPI `/workflow` request-response path

Entry path:

- `apps/fastapi/main.py` imports `app` from `langgraph_platform.api.main`.
- `create_app()` in `src/langgraph_platform/api/main.py` defines endpoint handlers.

Request schema (`WorkflowRequest`):

```
class WorkflowRequest(BaseModel):
    user_request: str = Field(..., min_length=1)
    session_id: str | None = None
```

Handler logic:

1. `result = engine.run(user_request=request.user_request, session_id=request.session_id)`
2. Return JSON dict:
   - `workflow_id`
   - `final_report`
   - `confidence`
   - `verification_status`

- `metadata` (serialized `ExecutionMetadata`)

Response shape (exact keys in code):

```
{
  "workflow_id": "wf_...",
  "final_report": "...",
  "confidence": 0.0,
  "verification_status": "passed|failed|needs_review|unknown",
  "metadata": {"workflow_id": "...", "session_id": "...", "node_status": {...}, "...": "...'
}
```

3.3 Flow C: Graph runtime internals (`LangGraphWorkflowEngine.run`)

Defined in `src/langgraph_platform/engine/workflow.py`.

Step-by-step:

1. `create_initial_state(user_request, session_id)` creates `WorkflowState` with:
   - `user_request`
   - `execution_metadata.workflow_id`
   - `execution_metadata.session_id`
2. `self.graph.invoke(initial.model_dump(mode="json"))`
3. `parse_node_result(raw)` converts dict back to `WorkflowState`.
4. Optionally logs metrics to MLflow if:
   - `ENABLE_MLFLOW=1`
   - `config.monitoring.mlflow_tracking_uri` exists.
5. Materializes `WorkflowResult`.

Conditional retry behavior:

- `_supervisor_route(...)` calls `should_retry_reflection(...)`.
- If confidence is below threshold and retries are below max, transition to `writer` again.
- Else transition to `finalize`.

3.4 Flow D: Node pipeline and state mutation semantics

All node handlers are in `src/langgraph_platform/engine/nodes.py`.

Common node lifecycle utilities:

- `_mark_node_start(runtime, state, node_name)`
- `_mark_node_end(runtime, state, node_name, failed=False)`
- `_persist_state(runtime, state, node_name)`
- `_serialize(state)` updates `execution_metadata.updated_at`

Planner node Function: `planner_node`

- Calls planner agent prompt via `runtime.llm_client.json_with_fallback(...)`.

- Writes:
  - `state.execution_plan`
  - `state.subtasks`
  - `state.routing` (from router + optional model routing override)
  - `state.intermediate_outputs["planner"]`
  - token counters in `state.token_usage`

Parallel research node Function: `parallel_research_node -> _parallel_research_sync`

- Builds callable list conditionally:
  - `duckduckgo_search` (if `state.routing.require_web_search`)
  - `github_search`
  - `documentation_search`
  - `memory_search` (if `state.routing.require_memory`)
- Executes tools in parallel using `threading.Thread` and `queue.Queue`.
- Aggregates into:
  - `state.search_results`
  - `state.citations`
  - `state.memory` (only dict results containing `workflow_id`)
- Runs RAG retrieval when `state.routing.require_rag`:
  - `runtime.rag_pipeline.retrieve(state.user_request, top_k=...)`

Knowledge merge + consensus

- `knowledge_merge_node`: deduplicates evidence by URL/title key and builds merged bullet context.
- `consensus_reasoning_node`: asks 3 models (`writer`, `researcher`, `verifier`) for drafts and keeps the longest candidate.

Writing + verification loop

- `writer_node`: generates markdown report; appends to `state.reports`; sets confidence floor.
- `fact_checker_node`: heuristic verification based on citation presence and report length.
- `reflection_node`: asks verifier model for `improvements`; appends `## Reflection Improvements` section.
- `critic_node`: rule-based risk checks (assumptions section, citation count).
- `qa_node`: checks report existence, failed verification, and confidence threshold.
- `citation_node`: deduplicates citations and appends a `## Citations` section to latest report.
- `supervisor_node`: decides `approve` vs `needs retry` using QA status, verification status, and threshold.
- `finalize_node`: persists terminal workflow record and timestamps `finished_at`.

3.5 Flow E: FastAPI auxiliary operational flows

`/chat` Request schema: `ChatRequest` (`message`, optional `session_id`).

Returns keys:

- `workflow_id`
- `response` (final report)
- `confidence`
- `verification_status`
- `citations` (list of serialized `Citation`)

`/memory`

- If `query` is provided: runs tool `memory_search` and returns `{"items": result.output}`.
- Else: returns recent workflow summaries from `SQLiteStore.list_recent_runs`.

`/reports` Request schema: `ReportExportRequest` with:

- `workflow_id`
- `markdown_report`
- `payload: dict[str, Any]`

Delegates to `ReportExporter.export_all(...)`, returning file paths for markdown/html/pdf/json.

`/knowledge` Request schema: `KnowledgeIngestRequest` with:

- `paths: list[str]`
- `urls: list[str]`

If paths exist: `engine.rag_pipeline.ingest_paths(paths)`. If URLs exist: `engine.rag_pipeline.ingest_urls(urls)`.

3.6 Flow F: Streamlit interactive runtime

Entry path:

- `apps/streamlit/app.py` -> `launch_dashboard()` in `src/langgraph_platform/ui/dashboard.py`.

UI pages and their code paths:

- `Dashboard`: `analytics.summary()`
- `Workflow Graph`: `engine.inspect_graph()` + `to_mermaid(...)`
- `Live Execution`: `engine.run(request)` on button click
- `Agents`: `list_agents()`
- `Shared State`: `sqlite_store.list_recent_runs(limit=10)`
- `Memory`: tool `memory_search`
- `Knowledge Base`: `rag_pipeline.ingest_paths/ingest_urls`
- `Reports`: run table from SQLite

- `Analytics`: Plotly charts from `AnalyticsService`
- `Configuration`: serialized config dump

3.7 Key data shapes to memorize

`WorkflowState` core fields From `src/langgraph_platform/state/models.py`:

- `user_request: str`
- `execution_plan: str`
- `subtasks: list[str]`
- `routing: RoutingDecision`
- `retrieved_documents: list[RetrievedDocument]`
- `search_results: list[dict[str, Any]]`
- `memory: list[dict[str, Any]]`
- `intermediate_outputs: dict[str, AgentOutput]`
- `reports: list[str]`
- `citations: list[Citation]`
- `verification_status: VerificationStatus`
- `confidence_score: float`
- `token_usage: TokenUsage`
- `execution_metadata: ExecutionMetadata`
- `tool_calls: list[ToolCallRecord]`
- `hitl_actions: list[HITLAction]`
- `paused: bool`

`RoutingDecision`

- `require_web_search: bool`
- `require_rag: bool`
- `require_memory: bool`
- `require_code_execution: bool`
- `require_verification: bool`

`ToolResult` From `src/langgraph_platform/tools/base.py`:

- `ok: bool`
- `output: Any`
- `source: str`
- `error: str | None`

SQLite tables created From `src/langgraph_platform/memory/sqlite_store.py` ORM models:

- `workflow_runs`
- `agent_outputs`
- `tool_calls`
- `graph_states`

Module 4: Setup & Run Guide

4.1 Prerequisites inferred from project files

From `pyproject.toml` and scripts:

- Python `>=3.12`
- `uv` package manager
- Local Ollama endpoint expected by default: `http://localhost:11434`
- Optional GPU metrics: NVML-compatible environment (`nvidia-ml-py`)

Core dependencies include:

- LangGraph, LangChain, FastAPI, Streamlit
- SQLAlchemy, ChromaDB, sentence-transformers
- Typer, Rich, Plotly
- HTTPX, BeautifulSoup4, Trafilatura
- WeasyPrint (for PDF export in report exporter)

4.2 Installation sequence on a clean machine

Typical setup commands (from repository conventions):

```
uv venv .venv
source .venv/bin/activate
uv sync
```

Optional developer extras (if needed):

```
uv sync --extra dev
```

4.3 Configuration files and precedence

Base file

- `configs/config.yaml`

Overlay files merged by loader

- `configs/models/default.yaml`
- `configs/graph/default.yaml`
- `configs/routing/default.yaml`
- `configs/retries/default.yaml`
- `configs/memory/default.yaml`
- `configs/embeddings/default.yaml`
- `configs/chunking/default.yaml`
- `configs/tools/default.yaml`
- `configs/analytics/default.yaml`
- `configs/mcp/default.yaml`
- `configs/hitl/default.yaml`
- `configs/prompts/default.yaml`

`load_config` behavior (`config/loader.py`):

1. Read `configs/config.yaml`.
2. Recursively read all `configs/**/*.yaml` except base.
3. Deep-merge dictionaries (`_merge_dict`).
4. Validate with `AppConfig`.
5. Apply environment overrides (`apply_env_overrides`).

4.4 Environment variables

From `.env.example` and `apply_env_overrides`:

Model/path overrides handled by code:

- `OLLAMA_PLANNER_MODEL`
- `OLLAMA_RESEARCHER_MODEL`
- `OLLAMA_WRITER_MODEL`
- `OLLAMA_VERIFIER_MODEL`
- `PLATFORM_SQLITE_PATH`

Documented optional provider keys:

- `WEATHER_API_KEY`
- `CURRENCY_API_KEY`
- `NEWS_API_KEY`

LLM behavior flags used in code (`engine/llm.py`):

- `OLLAMA_TIMEOUT_SECONDS` (overrides client timeout)
- `REQUIRE_LIVE_LLM=1` (disables fallback and raises if models fail)

Vector embedding behavior (`memory/vector_store.py`):

- `USE_SENTENCE_TRANSFORMERS=1` enables `SentenceTransformer` embeddings.

Optional run telemetry (`engine/workflow.py`):

- `ENABLE_MLFLOW=1` enables MLflow logging.

4.5 Typical start commands

API:

```
uv run uvicorn apps.fastapi.main:app --host 0.0.0.0 --port 8000 --reload
```

Dashboard:

```
uv run streamlit run apps/streamlit/app.py --server.port 8501 --server.address 0.0.0.0
```

CLI examples:

```
uv run langgraph-platform run "Analyze AI startup landscape"
uv run langgraph-platform graph
```

```
uv run langgraph-platform state --limit 20
uv run langgraph-platform memory "previous report"
uv run langgraph-platform doctor
```

Script wrappers present in `scripts/`:

- `scripts/run_api.sh`
- `scripts/run_dashboard.sh`
- `scripts/run_doctor.sh`

4.6 Database and persistence setup

No explicit migration tool is used in this repository.

Database initialization path:

- `LangGraphWorkflowEngine.__init__` creates `SQLiteStore` then calls `sqlite_store.init()`.
- `SQLiteStore.init()` runs `Base.metadata.create_all(self.engine)`.

Meaning:

- On first run, required tables are created automatically.
- No separate seeding step is required by default.

Chroma setup:

- `ChromaMemoryStore` initializes `chromadb.PersistentClient(path=...)`.
- Uses collection name: `knowledge`.
- In-memory fallback path exists if Chroma import/init fails.

4.7 External service expectations

From code paths in tools and LLM client, these integrations may be called at runtime:

- Ollama HTTP API (`/api/generate`)
- DuckDuckGo search/news (`ddgs` / `duckduckgo_search`)
- GitHub search API (`https://api.github.com/search/repositories`)
- Weather endpoint (`https://wttr.in/...`)
- URL fetcher for arbitrary web pages

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

Stage 1: Understand data contracts first Read in this order:

1. `src/langgraph_platform/state/models.py`
2. `src/langgraph_platform/config/settings.py`
3. `src/langgraph_platform/config/loader.py`

Goal:

- Understand what the system stores in state, and how configuration enters runtime.

Stage 2: Understand workflow orchestration Read in this order:

1. `src/langgraph_platform/engine/workflow.py`
2. `src/langgraph_platform/engine/router.py`
3. `src/langgraph_platform/engine/nodes.py`

Goal:

- Follow node-by-node control flow and retry logic.

Stage 3: Understand tooling and memory Read in this order:

1. `src/langgraph_platform/tools/base.py`
2. `src/langgraph_platform/tools/builtin.py`
3. `src/langgraph_platform/memory/sqlite_store.py`
4. `src/langgraph_platform/memory/vector_store.py`
5. `src/langgraph_platform/rag/pipeline.py`

Goal:

- Understand evidence retrieval and persistence behavior.

Stage 4: Understand interfaces (API/CLI/UI) Read in this order:

1. `src/langgraph_platform/api/schemas.py`
2. `src/langgraph_platform/api/main.py`
3. `src/langgraph_platform/cli/app.py`
4. `src/langgraph_platform/ui/dashboard.py`

Goal:

- Map external user actions to engine calls.

Stage 5: Understand observability and extension points Read in this order:

1. `src/langgraph_platform/monitoring/system.py`
2. `src/langgraph_platform/analytics/service.py`
3. `src/langgraph_platform/exporters/report_exporter.py`
4. `src/langgraph_platform/mcp/server.py`
5. `src/langgraph_platform/plugins/base.py` and `plugins/registry.py`

Goal:

- Understand metrics, exports, MCP, and plugin architecture.

5.2 Practical exercises (with solution outlines)

Exercise 1 Task:

- Trace exactly how `user_request` reaches the writer prompt in a CLI run.

What to inspect:

- `src/langgraph_platform/cli/app.py` (`run_workflow`)
- `src/langgraph_platform/engine/workflow.py` (`run`)
- `src/langgraph_platform/engine/nodes.py` (`writer_node`)

Solution outline:

- `run_workflow` passes CLI text into `engine.run(request)`.
- `engine.run` creates `WorkflowState(user_request=...)`.
- In `writer_node`, prompt includes `f"User request:\n{state.user_request}"`.

Exercise 2 Task:

- List the exact conditions for supervisor approval.

What to inspect:

- `src/langgraph_platform/engine/nodes.py` (`supervisor_node`)

Solution outline:

Approval requires all of:

- `qa_status == "passed"`
- `state.verification_status in {PASSED, UNKNOWN}`
- `state.confidence_score >= runtime.config.retry.confidence_threshold`

Exercise 3 Task:

- Explain how routing flags are computed and then overridden.

What to inspect:

- `src/langgraph_platform/engine/router.py` (`decide_routing`)
- `src/langgraph_platform/engine/nodes.py` (`planner_node`)

Solution outline:

- Initial flags come from keyword checks on `state.user_request`.
- If planner model returns `routing` dict, node overwrites each flag with returned booleans.

Exercise 4 Task:

- Describe where and how node timing is tracked.

What to inspect:

- src/langgraph_platform/monitoring/system.py
- src/langgraph_platform/engine/nodes.py helper functions

Solution outline:

- _mark_node_start calls monitor.start_node(node).
- _mark_node_end calls monitor.stop_node(node) and writes duration to state.execution_metadata.node_durations_ms[node].

Exercise 5 Task:

- Identify every place where workflow data is persisted to SQLite.

What to inspect:

- src/langgraph_platform/memory/sqlite_store.py
- src/langgraph_platform/engine/nodes.py

Solution outline:

- Per node snapshot: _persist_state -> save_workflow_state(...).
- Final summary: finalize_node -> finalize_workflow(...).
- Methods for agent output and tool call exist (save_agent_output, save_tool_call) though current node code does not call them directly.

Exercise 6 Task:

- Explain how RAGPipeline.retrieve converts Chroma output into typed objects.

What to inspect:

- src/langgraph_platform/rag/pipeline.py (retrieve)
- src/langgraph_platform/state/models.py (RetrievedDocument)

Solution outline:

- Reads first batch from dict keys: documents, metadatas, distances, ids.
- Iterates with zip(..., strict=False).
- Computes score = 1.0 - distance (clamped to [0.0, 1.0]).
- Returns list of RetrievedDocument.

Exercise 7 Task:

- Compare /workflow and /chat response payloads.

What to inspect:

- src/langgraph_platform/api/main.py

Solution outline:

- /workflow returns final_report, metadata.
- /chat returns response plus serialized citations.

- Both include `workflow_id`, `confidence`, `verification_status`.

Exercise 8 Task:

- Add a new tool mentally: where would you plug it in without touching workflow logic?

What to inspect:

- `src/langgraph_platform/tools/base.py`
- `src/langgraph_platform/tools/builtin.py`
- `src/langgraph_platform/agents/registry.py`

Solution outline:

- Implement class with `name` and `run(args, context)` returning `ToolResult`.
- Register it in `build_default_registry`.
- Add tool name in relevant agent spec under `AGENT_REGISTRY`.
- Workflow nodes can call it through `runtime.tool_registry.run("tool_name", args)`.

Learner Verification Checklist

Use this checklist to self-evaluate mastery:

- Can you explain the full graph path from `planner` to `finalize`, including retry routing from `supervisor`?
- Can you describe the complete structure of `WorkflowState` and why each section exists?
- Can you explain how configuration is layered (base YAML + overlays + env overrides)?
- Can you map each API endpoint to the exact backend function and return keys?
- Can you explain how search results, RAG documents, and citations are combined before writing?
- Can you explain the three-part quality loop: `fact_checker -> reflection -> critic -> qa`?
- Can you describe how and where SQLite and Chroma persistence are updated?
- Can you explain how `ToolRegistry` enables pluggable behavior without changing core workflow orchestration?
- Can you explain what happens when Ollama is unavailable and how fallback is controlled by env vars?
- Can you identify where to add a new agent, tool, or plugin entry point safely?